module 07: 状态机

1.状态机

- 使用Verilog设计电路，完成以下功能: 每瓶饮料1.5元，一次只能投入一个硬币，可投入0.5与1.0两种硬币，具有找零功能。

2.状态机检测原理

- 对于状态的区分，我们可以根据饮料机中剩余的的金额来进行分类，因为硬币只存在两种，0.5/1元，因此状态的划分可以如下
    - IDLE：复位状态，表示饮料机中的余额为0元
    - s1:饮料机的余额为0.5元
    - s2:饮料机中的余额为1元
    - s3:饮料机中的余额为1.5元（输出饮料，不找零）
    - s4:饮料集中的余额为2元（输出饮料，找零)

3.RTL

- 输入:两位input代表投入的硬币，input[1]拉高意味着投入一元，input[0]拉高意味着投入0.5元，默认一次只能投入一枚硬币。
- 输出:drink代表输出饮料，coin代表输出硬币。

```
module drink_machine( clk ,rst_n, money, drink, coin);

   input clk;
   input rst_n;
   input [1:0] money;
   output drink;
   output coin;

   parameter IDLE=3'd0, s1=3'd1, s2=3'd2, s3=3'd3, s4=3'd4;

   reg [2:0] state,nstate;

   always@(posedge clk or negedge rst_n)
   begin
    if(!rst_n)
       state<=IDLE;
    else
       state<=nstate;
   end

   always@(*)
   begin
    case(state)
       IDLE:nstate = money[1] ? s2 : (money[0] ? s1 :IDLE);
       s1:  nstate = money[1] ? s3 : (money[0] ? s2 : s1);
       s2:  nstate = money[1] ? s4 : (money[0] ? s3 : s2);
       s3:  nstate = money[1] ? s2 : (money[0] ? s1 :IDLE);
```

```
        s4:  nstate = money[1] ? s2 : (money[0] ? s1 :IDLE);
        default nstate = IDLE;
      endcase
    end

    assign drink = (state == s3 || state == s4)? 1:0;
    assign coin  = (state == s4)? 1:0;

endmodule
```

Moore vs Mealy 状态机

1.核心区别

对比项	Moore 状态机	Mealy 状态机
输出取决于	**仅当前状态**	**当前状态 + 当前输入**
输出更新时机	时钟边沿后（寄存器输出）	输入变化立即响应（组合逻辑输出）
达到同等功能所需状态数	较多	较少（通常少 1 个）
面积	状态多 → FF 略多，但输出逻辑简单	状态少 → FF 少，但输出组合逻辑更复杂
输出毛刺风险	**低**（输出来自 FF，天然干净）	**高**（输出是纯组合，随输入毛刺传播）
输出延迟	晚一拍（registered）	当拍即出（combinational）

> **一句话记忆**：Moore 输出看状态，Mealy 输出看状态+输入。

2.面积对比详解

Moore 面积更大的原因

- 为了把「输入影响」编码进状态，需要更多状态来区分「相同历史、不同当前输入」的情况
- 更多状态 → 需要更多 bit 的状态寄存器 → 更多 FF

Mealy 面积更小的原因

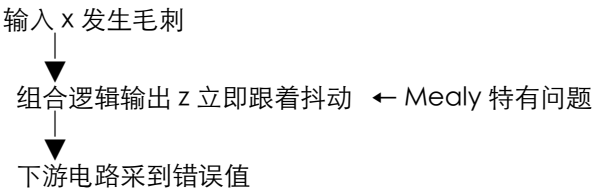
- 输出直接由 (state, input) 决定，不需要为每个输入情况单独建状态
- 状态数通常比等效 Moore 少一个或更多
- 但输出的组合逻辑扇入更大，可能抵消部分面积优势

结论

通常：Mealy FF 面积 < Moore FF 面积
但： Mealy 组合逻辑面积 ≥ Moore 组合逻辑面积
综合：小设计 Mealy 略占优；大设计差距不显著，timing 问题更值得关注

3.Timing 问题

3.1 Mealy 的主要问题：输出毛刺（Glitch）



- Mealy 输出是纯组合路径：z = f(state, x)
- 输入 x 的任何毛刺（glitch）都会**直接穿透**到输出
- 在异步接口或跨时钟域场景尤其危险

3.2 Mealy 的另一问题：输出路径时序紧张

FF(state) → 组合逻辑(next_state) → 同时驱动输出 z

- 输出 z 和次态逻辑共用同一组合路径
- 如果状态机复杂，这条路径可能成为关键路径，**限制最高工作频率**

3.3 Moore 的问题：输出延迟一拍

- Moore 输出来自寄存器，比输入晚一个时钟周期才能反映
- 在需要**当拍响应**的场景（如握手协议）会引入额外延迟

4.解决方案

4.1 解决 Mealy 毛刺：对输出加寄存器（Registered Output）

把 Mealy 的组合输出再打一拍，消除毛刺：

```
// Mealy 原始写法（有毛刺）
always @(*) begin
   case (state)
      S1: z = (x) ? 1'b1 : 1'b0;  // 组合输出，有毛刺
      default: z = 1'b0;
   endcase
end

// 修复写法：输出寄存一拍
always @(posedge clk) begin
   case (state)
```

```
      S1: z_reg <= (x) ? 1'b1 : 1'b0;  // 寄存后输出，无毛刺
      default: z_reg <= 1'b0;
   endcase
end
assign z = z_reg;
```

> ⚠ 代价：输出比原 Mealy 晚一拍，此时行为接近 Moore

4.2 解决 Mealy 关键路径：流水线 / 改用 Moore

- 将复杂的组合逻辑拆分，在中间插入寄存器（流水线）
- 或直接改用 Moore 写法，用更多状态换取更短的组合路径

4.3 解决 Moore 延迟一拍：预测输出（Look-ahead Output）

提前一拍根据**次态**计算输出，而非当前状态：

```
// 普通 Moore：输出晚一拍
assign z = (state == S_DONE);

// Look-ahead：根据 next_state 输出，当拍有效
assign z = (next_state == S_DONE);
```

> ⚠ 代价：next_state 是组合逻辑，输出路径变长，引入毛刺风险（与 Mealy 同理）

5.选型总结

场景	推荐
输出接异步电路 / 跨时钟域	Moore（或 Mealy + 寄存器输出）
需要减少状态数、节省面积	Mealy
需要当拍输出且无毛刺容忍	Moore + look-ahead（需权衡）
控制路径对 timing 敏感	Moore（输出路径短）
一般 RTL 设计默认推荐	Moore，更安全、更易 timing 收敛

6.RTL 对比示例（以序列检测 10 为例）

Moore 写法

```
module moore_10 (
   input  wire clk, rst_n, x,
   output wire z
);
   localparam S0=2'd0, S1=2'd1, S2=2'd2;  // 需要 3 个状态
```

```
    reg [1:0] state, next_state;

    always @(posedge clk) state <= rst_n ? next_state : S0;

    always @(*) begin
        case (state)
            S0: next_state = x ? S1 : S0;
            S1: next_state = x ? S1 : S2;  // 收到 0，进接受态
            S2: next_state = x ? S1 : S0;
            default: next_state = S0;
        endcase
    end

    assign z = (state == S2);  // 输出只看状态，无毛刺
endmodule
```

Mealy 写法

```
module mealy_10 (
    input  wire clk, rst_n, x,
    output reg  z
);
    localparam S0=1'd0, S1=1'd1;  // 只需 2 个状态，面积更小
    reg state, next_state;

    always @(posedge clk) state <= rst_n ? next_state : S0;

    always @(*) begin
        z = 1'b0;
        case (state)
            S0: next_state = x ? S1 : S0;
            S1: begin
                    next_state = x ? S1 : S0;
                    z = x ? 1'b0 : 1'b1;  // 输出依赖输入，有毛刺风险
                end
            default: next_state = S0;
        endcase
    end
endmodule
```